

**Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»**

**Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий**

**ОТЧЁТ
по лабораторной работе № 3 (часть 1)
по дисциплине
«Проектирование защищенных интеллектуальных
информационных систем»**

Тема: Анализ безопасности кода и дампов оперативной памяти

Выполнил:
студент группы 321701
Струнец Д. П.

Проверил:
Сальников Д. А.

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является разработка приложения с интерактивным интерфейсом для создания, обновления и удаления данных в оперативной памяти (публичных и конфиденциальных), защита конфиденциальных данных хешированием или шифрованием, а также анализ безопасности кода, потребления ресурсов и дампов памяти после create / update / delete.

2 КРАТКОЕ ОПИСАНИЕ РАЗРАБОТАННОГО ПРИЛОЖЕНИЯ

Приложение In-Memory Secure Store реализовано на Python 3 (Tkinter + библиотека cryptography). Все записи хранятся в словаре MemoryStore только в RAM процесса. Типы записей: public (открытый текст); confidential_hash (PBKDF2-HMAC-SHA256, необратимо); confidential_enc (Fernet/AES, обратимо ключом приложения). Предусмотрены валидация ввода, просмотр/расшифровка, вызов make_dump.sh и окно просмотра снимков из каталога dumps/.

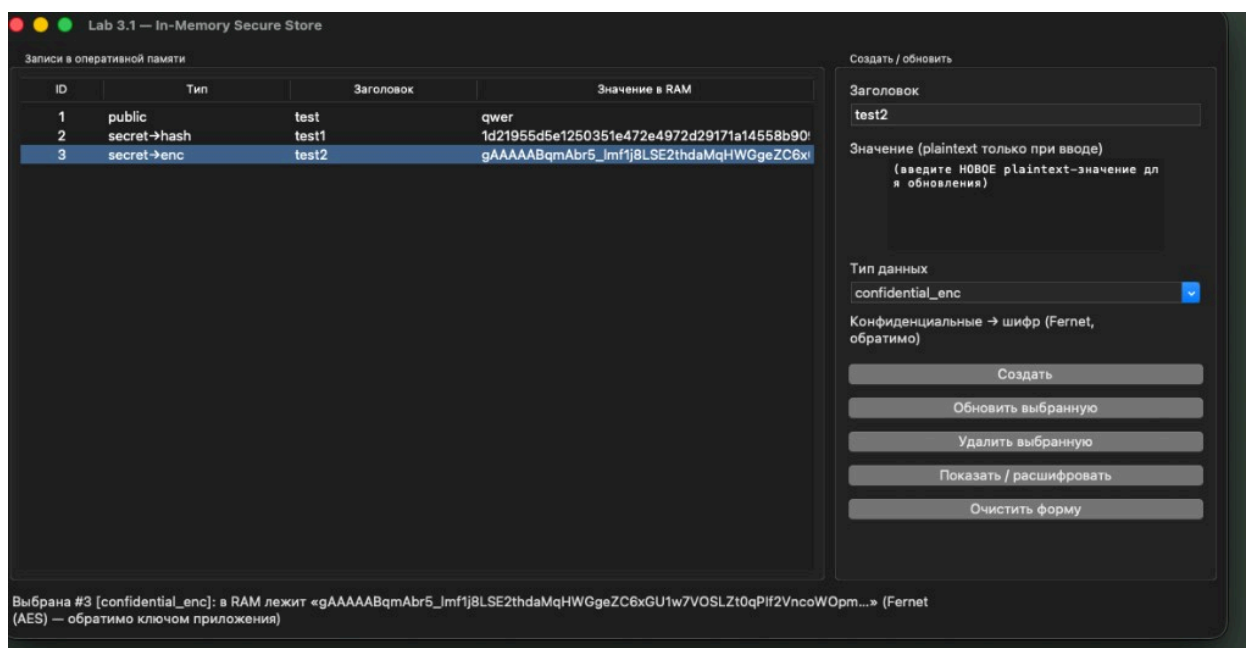


Рисунок 1 – Главное окно приложения (CRUD и типы защиты)

```

app.py - _protect()
def _protect(self, raw: str, kind: Kind) -> tuple[str, str, str]:
    if kind == Kind.PUBLIC:
        return raw, "", "хранится открытым текстом в RAM"
    if kind == Kind.CONFIDENTIAL_HASH:
        salt = secrets.token_bytes(16)
        digest = hashlib.pbkdf2_hmac(
            "sha256",
            raw.encode("utf-8"),
            salt,
            120_000,
        )
        stored = digest.hex()
        return stored, salt.hex(), "PBKDF2-HMAC-SHA256 (необратимо)"
    if kind == Kind.CONFIDENTIAL_ENC:
        if self._fernet is None:
            raise ValueError(
                "Для шифрования установите: pip install cryptography"
            )
        token = self._fernet.encrypt(raw.encode("utf-8")).decode("ascii")
        return token, "", "Fernet (AES) - обратимо ключом приложения"
    raise ValueError("Неизвестный тип записи")

class App(tk.Tk):
    def __init__(self) -> None:
        super().__init__()
        self.title("Lab 3.1 - In-Memory Secure Store")
        self.geometry("920x560")
        self.minsize(800, 480)

        self.store = MemoryStore()
        self._selected_id: Optional[int] = None

        self._build()
        self.refresh()

```

Рисунок 2 – Фрагмент кода защиты конфиденциальных данных

3 АНАЛИЗ ЗАЩИЩЁННОСТИ ПРИЛОЖЕНИЯ

С точки зрения кода приняты меры: разделение публичных и конфиденциальных данных; немедленное преобразование секрета в хеш или шифротекст при create/update; валидация пустых и чрезмерно длинных значений; сообщение об ошибке вместо падения; удаление записи из словаря с затиранием полей объекта.

Ограничения (важно для отчёта по безопасности):

- plaintext на короткое время существует в виджете Text и локальных str Python;
- ключ Fernet хранится в RAM процесса — дамп процесса theoretically компрометирует ciphertext;
- затирание строк в Python не гарантирует очистку всех копий в куче (GC/interning);
- полный core-дамп на macOS часто требует прав отладчика (SIP).

4 АНАЛИЗ ОПЕРАТИВНОЙ ПАМЯТИ И ПРОЦЕССОРНОГО ВРЕМЕНИ

Автотест выполнил create → dump → update → dump → delete → dump в одном процессе. По getrusage: CPU user ≈ 0,064 с, sys ≈ 0,065 с; max RSS ≈ 40 МБ (типично для интерпретатора Python + cryptography). Утилита

make_dump.sh дополнительно сохраняет ps/resources.txt и vmmap (карта регионов памяти). Полный файл core в тестовых прогонах на macOS не сформирован без elevated прав — для анализа использовались store_snapshot, RESULT и vmmap.

```
$ autotest create/update/delete + search
PID=61479
CREATE ok pub='PUBLIC_MARK_qwer' hash=2bb32c7e16ccf7740ec58822... enc=gAAAAABqmAuocDUOWZPpYIPpY2rX...
DUMP after_create exit=0 -> /Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_create_pid61479_202602_144232
UPDATE ok
DUMP after_update exit=0 -> /Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_update_pid61479_202602_144235
DELETE ok (had hash prefix 033041527b21ab063a1344b49ccab077...)
DUMP after_delete exit=0 -> /Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_delete_pid61479_202602_144238
SEARCH after_create: {"PUBLIC_MARK_qwer": ["store_snapshot.txt"], "PUBLIC_MARK_updated": [], "SECRET_PLAIN_hunter2": [], "SECRET_PLAIN_updated99": [], "CARD_PLAIN_4111111111111111": [], "gAAAAA": ["store_snapshot.txt"]}
SEARCH after_update: {"PUBLIC_MARK_qwer": [], "PUBLIC_MARK_updated": ["store_snapshot.txt"], "SECRET_PLAIN_hunter2": [], "SECRET_PLAIN_updated99": [], "CARD_PLAIN_4111111111111111": [], "gAAAAA": ["store_snapshot.txt"]}
SEARCH after_delete: {"PUBLIC_MARK_qwer": [], "PUBLIC_MARK_updated": ["store_snapshot.txt"], "SECRET_PLAIN_hunter2": [], "SECRET_PLAIN_updated99": [], "CARD_PLAIN_4111111111111111": [], "gAAAAA": []}
CPU user=0.064s sys=0.065s maxrss=40878080
```

Рисунок 3 – Протокол автотеста и поиск маркеров в снимках

5 АНАЛИЗ ДАМПОВ ОПЕРАТИВНОЙ ПАМЯТИ

Для трассировки вводились маркеры: PUBLIC_MARK_*, SECRET_PLAIN_*, CARD_PLAIN_*. Поиск выполнялся по текстовым артефактам снимка (в первую очередь store_snapshot.txt).

5.1 После создания данных

В снимке after_create: публичное значение PUBLIC_MARK_qwer присутствует открытым текстом; для пароля в store лежит 64 hex-символа PBKDF2, SECRET_PLAIN_hunter2 в snapshot не найден; для карты — ciphertext Fernet (префикс gAAAAA), CARD_PLAIN в snapshot не найден. Это соответствует ожидаемой модели хранения после _protect().

```

$ dump - after_create_pid61479_20260902_144232
===== RESULT.txt =====
OUT=/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_create_pid61479_20260902_144232
CORE=ok (/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_create_pid61479_20260902_144232/core)

===== meta.txt =====
label=after_create
pid=61479
time=2026-09-02T14:42:32+03:00
uname=Darwin Dmitrys-MacBook-Pro.local 25.5.0 Darwin Kernel Version 25.5.0: Mon Apr 27 20:41:26 PDT 2026; root:xnu-12377.121.6~2/RELEASE_ARM64_T8132 arm64
cmd=/opt/homebrew/Cellar/python@3.14/3.14.5/Frameworks/Python.framework/Versions/3.14/Resources/Python.app/Contents/MacOS/Python

===== resources.txt =====
61479 33264 435374848 0.0 0.2

===== store_snapshot.txt =====
pid=61479
records=3

#1 kind=public title=public_note salt= stored=PUBLIC_MARK_qwer
#2 kind=confidential_hash title=password salt=0c8e036a799c1f82c356740874delf59 stored=2bb32c7e16ccf7740e
8822b925a3edd8c3f0b51100f8e073ce59e912059b7e
#3 kind=confidential_enc title=card salt= stored=gAAAAABgmAuocDUOWZPpYIPpY2rXDGb1sEfa6x1wZqpCZ169OnZuKPl
7kQWYxs07mkhp57uVw2pDxWwKbs6jl9pelGV5cKskEEjyrsJ_S3tJoYPJsReBM=

```

Рисунок 4 – Снимок after_create (RESULT + store_snapshot)

5.2 После обновления данных

В after_update публичный маркер сменился на PUBLIC_MARK_updated; старый PUBLIC_MARK_qwer в snapshot отсутствует. Хеш и ciphertext пересчитаны заново; plaintext SECRET_PLAIN_updated99 / CARD_PLAIN в текстовых артефактах снимка не обнаружены. Вывод: store отражает актуальное защищённое представление, однако полный core мог бы сохранить «хвосты» старых строк в куче.

```

$ dump - after_update_pid61479_20260902_144235
===== RESULT.txt =====
OUT=/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_update_pid61479_20260902_144235
CORE=ok (/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_update_pid61479_20260902_144235/core)

===== meta.txt =====
label=after_update
pid=61479
time=2026-09-02T14:42:35+03:00
uname=Darwin Dmitrys-MacBook-Pro.local 25.5.0 Darwin Kernel Version 25.5.0: Mon Apr 27 20:41:26 PDT 2026; root:xnu-12377.121.6~2/RELEASE_ARM64_T8132 arm64
cmd=/opt/homebrew/Cellar/python@3.14/3.14.5/Frameworks/Python.framework/Versions/3.14/Resources/Python.app/Contents/MacOS/Python

===== resources.txt =====
61479 35664 435374848 0.0 0.2

===== store_snapshot.txt =====
pid=61479
records=3

#1 kind=public title=public_note salt= stored=PUBLIC_MARK_updated
#2 kind=confidential_hash title=password salt=d9614d1f14bd36589f93c7ecd5a770c7 stored=033041527b21ab063a144b49ccab0779a2ecda42add35abd3c40e320287301d
#3 kind=confidential_enc title=card salt= stored=gAAAAABqmAuocDUOWZPpYIPpY2rXDGb1sEefa6xlwZqpCZ169OnZuKPl7kQWYxs07mkhp57uVw2pDxWwKbs6jl9pelGV5cKskEEjyrsJ_S3TJoYPJsReBM=

```

Рисунок 5 – Снимок after_update

5.3 После удаления данных

После delete конфиденциальных записей в after_delete осталась одна публичная запись PUBLIC_MARK_updated; префикс gAAAAA и хеши секретов из snapshot исчезли. Это показывает корректное удаление из структуры приложения. Отдельно отмечено: отсутствие строк в snapshot ≠ гарантированное отсутствие остатков в физической RAM без полного core и анализа heap.

```

$ dump -- after_delete_pid61479_20260902_144238
===== RESULT.txt =====
OUT=/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_delete_pid61479_20260902_144238
CORE=ok (/Users/dmitry/Desktop/BSUIR/PSIIZ/sem_7/lab_3.1/dumps/after_delete_pid61479_20260902_144238/core)

===== meta.txt =====
label=after_delete
pid=61479
time=2026-09-02T14:42:38+03:00
uname=Darwin Dmitrys-MacBook-Pro.local 25.5.0 Darwin Kernel Version 25.5.0: Mon Apr 27 20:41:26 PDT 2026; root:xnu-12377.121.6~2/RELEASE_ARM64_T8132 arm64
cmd=/opt/homebrew/Cellar/python@3.14/3.14.5/Frameworks/Python.framework/Versions/3.14/Resources/Python.app/Contents/MacOS/Python

===== resources.txt =====
61479 35568 435374848 0.0 0.2

===== store_snapshot.txt =====
pid=61479
records=1

#1 kind=public title=public_note salt= stored=PUBLIC_MARK_updated

```

Рисунок 6 – Снимок after_delete

5.4 Хранение конфиденциальных и неконфиденциальных данных

- Неконфиденциальные: plaintext в RAM и в snapshot — ожидаемо и подтверждено поиском маркера.
- Конфиденциальные (хеш): в store только PBKDF2; исходник в snapshot после create не найден.
- Конфиденциальные (шифр): в store Fernet; расшифровка возможна только с ключом процесса.
- Удаление убирает объекты из словаря; для полной санации RAM нужны secure memory / языки с ручным управлением буферами.

6 ВЫВОД

Семантическая составляющая. Реализовано UI-приложение с CRUD в оперативной памяти, разделением типов данных и защитой секретов хешированием/шифрованием; выполнен анализ кода, ресурсов и трёх снимков памяти по сценарию задания.

Прагматическая составляющая. Автотест и make_dump.sh дают воспроизводимые артефакты для демонстрации: публичные данные читаются из snapshot, секреты — нет (в store); удаление очищает логическое хранилище. Для усиления защиты рекомендуется минимизировать lifetime plaintext, не держать ключ рядом с данными дольше необходимого и

снимать core на Linux VM с gcore для более глубокого анализа остатков в heap.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ховард, М. Защищённый код / М. Ховард, Д. Лебланк. – М. : Русская редакция, 2004. – 704 с.
2. Ховард, М. 24 смертных греха компьютерной безопасности / М. Ховард, Д. Лебланк, Дж. Вьегга. – СПб. : Питер, 2010. – 400 с.
3. Cryptography Fernet [Электронный ресурс]. – Режим доступа: <https://cryptography.io/en/latest/fernet/>.
4. Apple Developer Documentation — vmmap [Электронный ресурс].